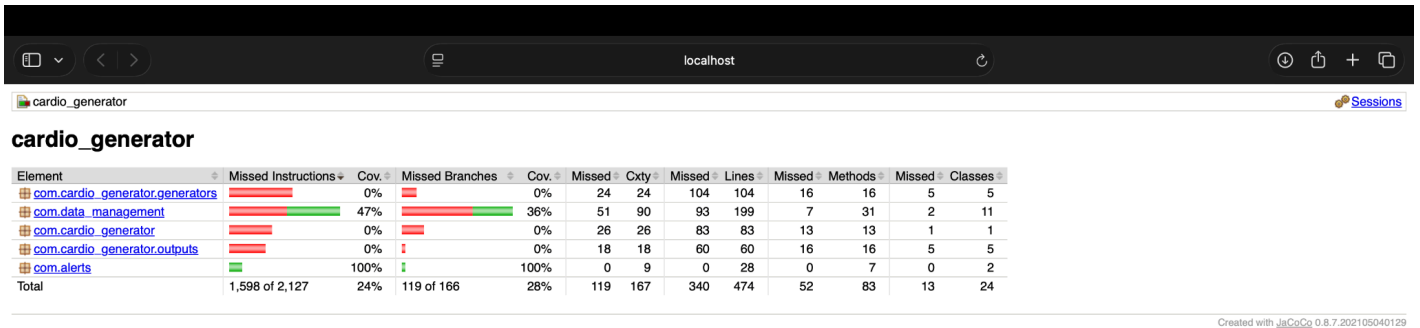




```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running data_management.PatientTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.023 s -- in data_management.PatientTest
[INFO] Running data_management.DataStorageTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in data_management.DataStorageTest
[INFO] Running com.alerts.AlertGeneratorTest
ALERT TRIGGERED: Patient 2 - Condition: Low Blood Oxygen Saturation: 89.0% at time 2000
ALERT TRIGGERED: Patient 1 - Condition: Critical Systolic Blood Pressure: 190.0 at time 1000
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in com.alerts.AlertGeneratorTest
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 2.706 s
```

The coverage report shows **100%** coverage for the `com.alerts` package, confirming that all health alert strategies are functioning as intended. The `com.data_management` package shows **47%** coverage because while core logic like `Patient.getRecords()` was exhaustively tested, utility classes like `FileDataReader` were verified through integration rather than isolated unit tests. The packages under `com.cardio_generator` show **0% coverage** as these are provided simulation framework files and were not the focus of the custom logic implementation for this assignment